

Macro Project 1 — Data Suite

Study Sheet

Contents

1	Macro Project 1 — Data Suite (Study Sheet)	2
1.1	Quickstart (how to run)	2
1.2	High-level architecture (what runs where)	2
1.3	Common artifact schemas (what columns to expect)	3
1.4	End-to-end data flow diagram	3
1.5	Core “mental model” types	3
1.6	How to read the function write-ups below	4
2	Concepts (classroom + intuitive)	4
2.1	Forecasting	4
2.2	Time-series transforms	7
2.3	Technical indicators	8
2.4	Macro transforms	10
2.5	Econometrics	12
2.6	News sentiment	14
3	Function-by-function reference	14
3.1	streamlit_app.py (UI, orchestration, rendering)	14
3.2	scripts/e2e_smoke.py (smoke test entrypoint)	17
3.3	src/finrec/config.py (configuration)	17
3.4	src/finrec/jobs/runner.py (job execution)	18
3.5	src/finrec/storage/sqlite.py (job persistence)	18
3.6	src/finrec/providers/** (data providers)	19
3.6.1	providers/utils/optional.py (optional dependency gating)	19
3.6.2	providers/registry.py (dynamic provider registration)	20
3.6.3	providers/market/yfinance.py (Yahoo Finance via yfinance)	20
3.6.4	providers/market/fmp.py (Financial Modeling Prep, with fallback)	21
3.6.5	providers/macro/fred.py (FRED via pandas-datareader)	21
3.6.6	providers/news/gdelt.py (GDELT doc API)	21
3.7	src/finrec/ui/**, src/finrec/viz/**, src/finrec/datasets/**	22
3.7.1	ui/pipelines.py (turn providers/recipes into jobs)	22
3.7.2	ui/timeseries_catalog.py (curated option lists)	22
3.7.3	viz/plotly_timeseries.py (Plotly figure construction)	23
3.7.4	datasets/merge.py (artifact merging)	23
3.8	src/finrec/recipes/** (transformations and models)	23

3.8.1	Common recipe contract (what every recipe does)	23
3.8.2	<code>recipes/registry.py</code> (recipe registration)	23
3.8.3	<code>recipes/forecast/_utils.py</code> (forecasting utilities and standardized schema)	24
3.8.4	Forecast recipes (all output the <code>build_forecast_output</code> schema)	25
3.8.5	Time-series transforms	26
3.8.6	Technical indicators	26
3.8.7	Macro transforms	26
3.8.8	Econometrics	27
3.8.9	News sentiment (FinBERT)	27

1 Macro Project 1 — Data Suite (Study Sheet)

Local-first Streamlit app for pulling/transforming **market**, **macro**, and **news** data; running **recipes** (indicators, econometrics, forecasting, sentiment); and persisting results as **jobs** with **SQLite logs** and **CSV artifacts**.

1.1 Quickstart (how to run)

- Create venv, install editable package (and extras as needed), copy `.env`, run Streamlit:
- `pip install -e ".[dev]"` then `streamlit run streamlit_app.py`
- Optional feature extras (install as needed):
- Market: `pip install -e ".[market]"`
- Macro: `pip install -e ".[macro]"`
- News: `pip install -e ".[news]"`
- Forecast: `pip install -e ".[forecast]"`
- Econometrics: `pip install -e ".[econ]"`
- ML (FinBERT): `pip install -e ".[ml]"`

1.2 High-level architecture (what runs where)

- **UI layer:** `streamlit_app.py` renders controls, submits jobs, and renders results by reading job status + artifacts.
- **Job layer:** `JobRunner` runs work in a background thread; `SQLiteStorage` persists job lifecycle + logs.
- **Providers:** `Provider.fetch()` pulls raw data (market/macro/news) into a normalized dataframe schema.
- **Recipes:** `Recipe.run()` transforms an input dataframe (indicators, econometric models, forecasts, sentiment aggregation).
- **Artifacts:** successful jobs write a CSV to `FINREC_RESULTS_DIR` and store its path in SQLite.

1.3 Common artifact schemas (what columns to expect)

- **Market providers** ('kind='market'): date (YYYY-MM-DD string), symbol, and numeric OHLCV columns like open/high/low/close/volume (some may be missing depending on provider/interval).
- **Macro providers** ('kind='macro'): long-form date, series_id, value (numeric).
- **News providers** ('kind='news'): ts (timestamp string), date, query, language, title, snippet, source, url.
- **Forecast recipes**: long-form rows with date, y, yhat, segment in {train,test,forecast}, method, target (optional yhat_lower/yhat_upper).
- **Econometrics recipes (OLS/AR1)**: coefficient table with term, coef, std_err, t, p_value, plus nobs, r2, adj_r2.
- **LP-IRF recipe**: horizon table with horizon, irf, std_err, ci_low, ci_high, nobs.

1.4 End-to-end data flow diagram

flowchart TD

```
user[User] --> streamlitUI[Streamlit_UI]
streamlitUI --> sessionState[SessionState_singletons]
streamlitUI --> submitProvider[submit_provider_fetch]
streamlitUI --> submitRecipe[submit_recipe_run]

submitProvider --> jobRunner[JobRunner_submit]
submitRecipe --> jobRunner

jobRunner --> sqlite[SQLiteStorage_jobs_logs]
jobRunner --> artifact[CSV_artifact_results_dir]

jobRunner --> providerFetch[Provider_fetch]
jobRunner --> recipeRun[Recipe_run]

providerFetch --> providerRegistry[get_registry]
providerRegistry --> providerImpl[Provider_impl_fetch]

recipeRun --> recipeRegistry[get_recipe_registry]
recipeRegistry --> recipeImpl[Recipe_impl_run]

sqlite --> streamlitUI
artifact --> streamlitUI
```

1.5 Core “mental model” types

- **Provider**** (src/finrec/providers/base.py): fetch(request, ctx) -> pd.DataFrame
- **Recipe**** (src/finrec/recipes/base.py): run(df, params, ctx) -> pd.DataFrame
- **JobRunner**** (src/finrec/jobs/runner.py): submit(..., fn) runs fn(ctx) in a thread, persists status/logs, writes CSV artifact.
- **SQLiteStorage**** (src/finrec/storage/sqlite.py): job table + job_logs table.

1.6 How to read the function write-ups below

Each function entry includes:

- **Technical:** inputs/outputs, side effects, algorithm steps, error handling, optional dependencies.
- **Intuitive:** plain-English purpose (“why this exists”).
- **Gotchas:** common edge cases and failure modes.

2 Concepts (classroom + intuitive)

This section is meant to read like class notes: each method has a compact formula/definition, a “when to use it,” and a short plain-English explanation. The function reference later is the implementation-level detail.

2.1 Forecasting

Naive forecast (`forecast_naive`)

Classroom explanation

Definition. Predict every future value with the last observed training value:

$$\hat{y}_{t+h} = y_t \quad \text{for } h = 1, 2, \dots$$

Use when. You need a baseline to beat, or the series is roughly a random walk.

Intuitive explanation

“Assume nothing changes. Whatever the series is today is the best guess for tomorrow.”

Where you use it in this project

Recipe: `NaiveForecastRecipe (id='forecast_naive')`. Output is normalized via `build_forecast_output`.

Drift forecast (forecast_drift)

Classroom explanation

Definition. Extend a straight line from the first to last training point:

$$\hat{y}_{t+h} = y_t + h \cdot \frac{y_t - y_1}{t - 1}.$$

Use when. The series has a clear trend and you want a simple trending baseline.

Intuitive explanation

“Keep moving forward at the average pace you’ve been moving.”

Where you use it in this project

Recipe: `DriftForecastRecipe (id='forecast_drift')`.

ETS / Holt–Winters exponential smoothing (forecast_ets)

Classroom explanation

Core idea. Build forecasts from exponentially-weighted components (level, trend, and optionally seasonality). A simple level-only form is:

$$\ell_t = \alpha y_t + (1 - \alpha)\ell_{t-1}, \quad \hat{y}_{t+h} = \ell_t.$$

Use when. You want a strong classic method for smooth series with level/trend/seasonality (especially at regular frequencies).

Intuitive explanation

“Smooth out the noise, keep the main pattern, then extend that pattern forward.”

Where you use it in this project

Recipe: `ETSForecastRecipe (id='forecast_ets'),` using `statsmodels.tsa.holtwinters`.

ARIMA(p, d, q) (forecast_arima)

Classroom explanation

Model. ARIMA combines autoregression (AR), differencing (I), and moving average (MA). A standard compact form is:

$$\phi(L)(1 - L)^d y_t = c + \theta(L)\varepsilon_t,$$

where L is the lag operator, $p = \deg(\phi)$, d is the differencing order, $q = \deg(\theta)$, and ε_t is a white-noise shock. **Use when.** You have a single time series with persistence; differencing makes it approximately stationary; you want interpretable, classical forecasting with uncertainty intervals when available.

Intuitive explanation

“Predict the next value using patterns in recent history—after you remove the big trend so the model is learning the ‘usual wiggles.’”

Where you use it in this project

Recipe: `ARIMAForecastRecipe` (`id='forecast_arima'`), implemented with `statsmodels` SARIMAX and a small AIC grid search over (p, d, q) .

Lag-feature ML forecasting (Ridge / Random Forest)

Classroom explanation

Setup. Turn a time series into supervised learning by using the last k lags as features:

$$X_t = (y_{t-1}, y_{t-2}, \dots, y_{t-k}), \quad \text{predict } y_t.$$

Train a model (Ridge regression or Random Forest) and forecast multiple steps via recursive prediction. **Use when.** You want a simple ML baseline that can capture nonlinearities (RF) or stable linear effects (Ridge) without specifying an ARIMA structure.

Intuitive explanation

“Teach a model: ‘given what happened in the last k periods, what usually happens next?’ Then let it roll forward step by step.”

Where you use it in this project

Recipes: `RidgeLagForecastRecipe` (`id='forecast_ridge_lags'`) and `RandomForestLagForecastRecipe` (`id='forecast_rf_lags'`).

2.2 Time-series transforms

Log returns (log_returns)

Classroom explanation

Definition. For a price/index $p_t > 0$,

$$r_t = \log(p_t) - \log(p_{t-1}).$$

For small changes, $r_t \approx \frac{p_t - p_{t-1}}{p_{t-1}}$. **Use when.** You want growth rates that are more stable than raw levels; common in finance and macro.

Intuitive explanation

“Instead of tracking the level, track the percent-ish change from one period to the next.”

Where you use it in this project

Recipe: `LogReturnsRecipe (id='log_returns')`.

Rolling z-score (rolling_zscore)

Classroom explanation

Definition. With rolling mean μ_t and std σ_t ,

$$z_t = \frac{x_t - \mu_t}{\sigma_t}.$$

Use when. You want a standardized “how extreme is this?” signal relative to recent history.

Intuitive explanation

“How many ‘usual’ units above or below normal is today, compared to the recent past?”

Where you use it in this project

Recipe: `RollingZScoreRecipe (id='rolling_zscore')`.

2.3 Technical indicators

Simple moving average (SMA) (sma)

Classroom explanation

$$\text{SMA}_t = \frac{1}{w} \sum_{i=0}^{w-1} x_{t-i}.$$

Use when. You want a smooth trend line that filters short-run noise.

Intuitive explanation

“Average the last w points so the line doesn’t jump around as much.”

Where you use it in this project

Recipe: `SimpleMovingAverageRecipe (id='sma')`.

Exponential moving average (EMA) (ema)

Classroom explanation

$$\text{EMA}_t = \alpha x_t + (1 - \alpha) \text{EMA}_{t-1}, \quad \alpha = \frac{2}{\text{span} + 1}.$$

Use when. You want smoothing that reacts faster to recent changes than SMA.

Intuitive explanation

“Like an average, but it ‘listens’ more to what just happened.”

Where you use it in this project

Recipe: `ExponentialMovingAverageRecipe (id='ema')`.

RSI (Relative Strength Index) (rsi)

Classroom explanation

Definition. Compare average gains to average losses over a window:

$$RS = \frac{\text{avg gain}}{\text{avg loss}}, \quad RSI = 100 - \frac{100}{1 + RS}.$$

Use when. You want a 0–100 momentum oscillator often used to summarize “recent up vs down pressure.”

Intuitive explanation

“In the last few periods, were moves mostly up or mostly down—and by how much?”

Where you use it in this project

Recipe: `RSIRecipe (id='rsi')`.

MACD (macd)

Classroom explanation

Definition. $\text{MACD line} = \text{EMA}_{\text{fast}} - \text{EMA}_{\text{slow}}$. $\text{Signal line} = \text{EMA}(\text{MACD line})$. $\text{Histogram} = \text{difference}$. **Use when.** You want a compact trend/momentum summary based on two smoothing speeds.

Intuitive explanation

“Compare a fast trend line to a slow trend line. When the fast one pulls away, momentum is changing.”

Where you use it in this project

Recipe: `MACDRecipe (id='macd')`.

Rolling volatility (rolling_vol)

Classroom explanation

Definition. Compute returns (often percent change), take rolling standard deviation σ_t , and optionally annualize:

$$\sigma_t^{(\text{ann})} = \sigma_t \sqrt{m},$$

where m is periods per year (e.g., 252 trading days). **Use when.** You want a moving measure of variability/risk.

Intuitive explanation

“How bumpy has this series been lately?”

Where you use it in this project

Recipe: `RollingVolatilityRecipe (id='rolling_vol')`.

2.4 Macro transforms

Inflation YoY (inflation_yoy)

Classroom explanation

$$\pi_t^{\text{YoY}} = 100 \left(\frac{P_t}{P_{t-12}} - 1 \right).$$

Use when. You want “inflation over the last year” from an index like CPI.

Intuitive explanation

“Compare today’s price index to the same month last year.”

Where you use it in this project

Recipe: `InflationYoYRecipe (id='inflation_yoy')`. Assumes monthly frequency (12 lags).

Inflation MoM annualized (inflation_mom_ann)

Classroom explanation

$$\pi_t^{\text{MoM, ann}} = 1200(\log P_t - \log P_{t-1}).$$

Use when. You want the one-month inflation rate expressed at an annual pace.

Intuitive explanation

“Take this month’s change and ask: if that kept happening every month, what yearly inflation would that imply?”

Where you use it in this project

Recipe: `InflationMoMAnnRecipe (id='inflation_mom_ann')`. Requires $P_t > 0$.

Growth QoQ annualized (growth_qoq_ann)

Classroom explanation

$$g_t^{\text{QoQ, ann}} = 400(\log X_t - \log X_{t-1}).$$

Use when. You want quarterly growth expressed at an annual rate (typical for GDP-style reporting).

Intuitive explanation

“Turn the quarter’s growth into a ‘per-year’ number so it’s easier to compare.”

Where you use it in this project

Recipe: `GrowthQoQAnnRecipe (id='growth_qoq_ann')`. Assumes adjacent rows are quarters.

Spread (spread) and Real rate (real_rate)

Classroom explanation

Spread. $s_t = a_t - b_t$. **Real rate (ex-post approx).** $r_t^{\text{real}} = i_t - \pi_t$. **Use when.** Spreads are common in term-structure/macro; real rates adjust nominal rates for inflation.

Intuitive explanation

“A spread is just ‘this minus that.’ A real rate is ‘interest minus inflation.’”

Where you use it in this project

Recipes: `SpreadRecipe (id='spread')` and `RealRateRecipe (id='real_rate')`.

2.5 Econometrics

OLS regression (ols)

Classroom explanation

Model. $y = X\beta + \varepsilon$. OLS estimates $\hat{\beta}$ by minimizing $\sum_t \varepsilon_t^2$. **Use when.** You want a baseline linear “what predicts/explains y ?” model with interpretable coefficients.

Intuitive explanation

“Fit the best straight-line relationship between y and the variables you think matter.”

Where you use it in this project

Recipe: `OLSRecipe (id='ols')` using `statsmodels`.

AR(1) / persistence regression (ar1)

Classroom explanation

$$y_t = c + \phi y_{t-1} + \varepsilon_t.$$

Use when. You want a simple persistence estimate; ϕ near 1 means very persistent.

Intuitive explanation

“Does yesterday’s value do most of the work in predicting today?”

Where you use it in this project

Recipe: `AR1Recipe (id='ar1')`.

Local projections IRF (LP-IRF) (lp_irf)

Classroom explanation

Idea. For each horizon h , run:

$$y_{t+h} = \alpha_h + \beta_h \text{shock}_t + \gamma_h^\top \text{controls}_t + u_{t+h}.$$

The impulse response at horizon h is β_h , and plotting $\{\beta_h\}_{h=0}^H$ traces the dynamic effect of a shock. **Use when.** You want a flexible IRF without committing to a full VAR system.

Intuitive explanation

“Ask a sequence of questions: if there’s a shock today, what happens to y next period? two periods out? three?”

Where you use it in this project

Recipe: `LocalProjectionIRFRecipe (id='lp_irf')`.

2.6 News sentiment

FinBERT daily sentiment (news_sentiment)

Classroom explanation

Mechanism. FinBERT classifies finance text into labels (POS/NEG/NEU) with confidence scores. This project maps labels to a signed score and aggregates to daily summaries (mean/sum, counts, and label shares). **Use when.** You want a daily sentiment time series from a set of news articles.

Intuitive explanation

“Have a model read a bunch of headlines, score how positive vs negative they sound, then average that into one number per day.”

Where you use it in this project

Recipe: `FinBERTDailySentimentRecipe (id='news_sentiment')`, using transformers with ProsusAI/finbert.

3 Function-by-function reference

3.1 streamlit_app.py (UI, orchestration, rendering)

- `_get_singletons()`
- **Technical:** Initializes long-lived objects in `st.session_state` (config, SQLiteStorage, JobRunner) plus a bunch of UI default flags/fields; returns (cfg, storage, runner).
- **Intuitive:** Streamlit reruns the script on every interaction; this function keeps “the app’s brain” from resetting each rerun.
- **Gotchas:** If you change `.env` values you usually need to restart Streamlit so `load_config()` takes effect; schema init happens once per session.
- `_parse_language_list(raw)`
- **Technical:** Parses comma-separated input into normalized lowercased language tokens for GDELT `sourcelang`: filters.
- **Intuitive:** Turns “english, spanish” into `["english", "spanish"]`.
- **Gotchas:** No validation against GDELT’s supported language names.
- `_df_with_rank(df, start=1)`
- **Technical:** Copies `df` and inserts a “#” column with a 1-based integer rank for display.
- **Intuitive:** Adds a “row number” column without mutating the original dataframe.
- **Gotchas:** Rank is purely positional; it doesn’t sort.
- `_cols_with_visibility_selector(cols, max_visible=3, displayed_count=2, session_key, label=...)`

- **Technical:** If too many series are selected, uses a Streamlit multiselect to choose `displayed_count` of them; returns (`chosen`, `remaining`) for plotting + table.
- **Intuitive:** Prevents charts from getting unreadably crowded when many series are selected.
- **Gotchas:** Ensures at least one chosen column (falls back to the default).
- `_parse_csv_list(raw)`
- **Technical:** Comma-splits, trims, uppercases; drops empty items.
- **Intuitive:** Standardizes ticker / series-id lists typed by the user.
- **Gotchas:** Uppercasing is good for tickers/FRED ids but may be surprising for other strings.
- `_update_recent_list(recent, used_ids, max_len=20)`
- **Technical:** Implements LRU behavior: for each used id, remove existing occurrence then insert at front; truncates to `max_len`.
- **Intuitive:** “Recently used series” stay at the top of selection lists.
- **Gotchas:** Mutates `recent` in place (which is fine because it’s a `session_state` list).
- `_auto_news_query_finance(tickers)`
- **Technical:** Builds a minimal GDELT query like (AAPL OR "Apple" OR MSFT OR "Microsoft") using a small expansion map and deduping.
- **Intuitive:** Saves you from writing the news query manually for common tickers.
- **Gotchas:** Expansion map is partial; query is intentionally simple (may over/under-match).
- `_auto_news_query_econ(series_ids)`
- **Technical:** Similar to finance, but maps common FRED ids to plain-English macro topics (e.g., CPIAUCSL → `inflation`) and ORs them.
- **Intuitive:** Converts “which macro series I’m studying” into “what to search in news.”
- **Gotchas:** Mapping is heuristic; users can override in the UI.
- `_job_by_id(storage, job_id)`
- **Technical:** Linear search over `storage.list_jobs(limit=1000)` to find a matching `job_id`.
- **Intuitive:** Fetch one job row when you only have its id.
- **Gotchas:** If there are >1000 jobs, the job may be missing from the limited list.
- `_artifact_df(path)`
- **Technical:** Thin wrapper around `pd.read_csv(path)` for job artifacts.
- **Intuitive:** “Load the output CSV for a finished job.”
- **Gotchas:** No schema checking; exceptions are handled by callers.
- `_download_df(df, label, file_name)`
- **Technical:** Uses `st.download_button` to offer a CSV download of the dataframe.
- **Intuitive:** One-click “export this table.”
- **Gotchas:** CSV is encoded as UTF-8 bytes; large frames can be slow.
- `_parse_cols(raw)`
- **Technical:** Comma-splits and trims into a list of column names.
- **Intuitive:** Lets users type CPIAUCSL, UNRATE to specify columns.
- **Gotchas:** Case-sensitive matching later depends on dataframe column names.
- `_preferred_market_provider(market_provider_ids)`
- **Technical:** Chooses `"fmp"` if present *and* `FINREC_FMP_API_KEY` is set, else prefers `"yfinance"`, else first id.
- **Intuitive:** Defaults to the most reliable market source available.

- **Gotchas:** Only checks presence of an API key string, not whether it’s valid.
- `_filter_df_by_date_range(df, x_col, start_date, end_date)`
- **Technical:** Converts `df[x_col]` to `datetime`; filters rows whose `.dt.date` is in `\{([start,end])\}`; returns a copy.
- **Intuitive:** Clips charts/tables to the date range you selected.
- **Gotchas:** If `x_col` can’t be parsed, returns original `df` unchanged.
- `_split_cols_for_secondary_axis_by_scale(df, cols, ratio_threshold=50.0)`
- **Technical:** Computes median absolute magnitude per column; picks the largest-magnitude column as “base”; assigns columns to right axis if `\{(\text{base}/\text{col})\} \ge \{threshold\}`.
- **Intuitive:** Prevents “tiny” series from looking flat when plotted with huge-valued series.
- **Gotchas:** Heuristic—can misclassify when series have outliers or near-zero medians.
- `_plot_df(df, x_col, left_cols, right_cols_override=None, ..., start_date=None, end_date=None)`
- **Technical:** Applies date filtering (optional), determines right-axis columns either from `right_cols_override` or the UI “secondary axis” list, then calls `plot_timeseries()` and renders with `st.plotly_chart`.
- **Intuitive:** One standardized way to “take a dataframe and show a nice time-series chart.”
- **Gotchas:** If no chartable columns exist, it prints a caption and returns (no error).
- `_plot_lp_irf(df_out, height=250, title=...)`
- **Technical:** Validates LP-IRF output columns (`horizon, irf, ci_low, ci_high`), builds a Plotly figure with a filled confidence band and IRF line.
- **Intuitive:** Visualizes an impulse response function with uncertainty bands.
- **Gotchas:** Requires Plotly optional dependency; silently declines to plot if required columns are missing.
- `_wide_from_market_artifacts(dfs_by_symbol, date_col='date', value_col='close', how='outer')`
- **Technical:** For each symbol, extracts `[date,value]`, renames `value` to `{SYM}_{value_col}`, merges into a “wide” table on date.
- **Intuitive:** Turns many per-ticker CSVs into one merged dataframe for charting/comparison.
- **Gotchas:** Date parsing failures drop rows; missing symbols/columns are skipped.
- `_wide_from_macro_artifacts(dfs_by_series, date_col='date', value_col='value', how='outer')`
- **Technical:** Same wide-merge idea for macro series, then forward-fills each value column so mixed-frequency series plot continuously.
- **Intuitive:** Makes monthly CPI and weekly claims readable on the same merged chart.
- **Gotchas:** Forward-fill is a modeling choice (can be misleading if interpreted as “real” observations).
- `_dataset_builder_ui(cfg, storage, runner, reg, start_date, end_date, default_fred_series, default_tickers)`
- **Technical:** UI to fetch multiple macro/market series as jobs, track their completion, then submit a merge job that runs `build_merged_dataset(...)`.
- **Intuitive:** A guided workflow to build a single “analysis-ready” dataset without leaving the app.
- **Gotchas:** Uses many `session_state` keys; relies on artifacts existing before merge is enabled.
- **Inner def `_merge_job_fn(ctx)`:** logs merge parameters, calls `build_merged_dataset`, logs output shape, returns merged dataframe for artifact writing.
- `_render_query_and_submit(mode, cfg, storage, runner, reg, start_date, end_date)`

- **Technical:** Renders the top-half “inputs” UI for Finance/Econ modes, plus News options; on “Run” it submits provider jobs (market or macro, plus optional news) and stores a structured `latest_run` dict in `session_state`.
- **Intuitive:** The “control panel” that turns user choices into background jobs.
- **Gotchas:** Many branches; correctness depends on consistent request schemas expected by providers/recipes.
- **`_render_results(storage, runner, start_date, end_date, recession_intervals)`**
- **Technical:** Reads `latest_run`, lists recent jobs from SQLite, conditionally auto-refreshes while jobs are RUNNING, loads artifacts for SUCCEEDED jobs, and renders tables/charts; also triggers downstream recipe jobs (indicators/forecasts/models/sentiment) once inputs are ready.
- **Intuitive:** The “dashboard” that watches jobs finish and shows outputs as they appear.
- **Gotchas:** Job tables are paged (`limit=500`); artifacts might fail to load if files moved/deleted.
- **Inner def `_forecast_metrics(df_fcst)`:** computes MAE/RMSE/MAPE on `segment=='test'` rows (requires `y`, `yhat`, `segment`).
- **Inner def `_iter_active_job_ids()`:** collects all job ids referenced by `latest_run` and dataset-builder/usrec session keys; de-dupes.
- **Inner def `_is_job_active(job_id)`:** returns True if job exists and status is RUNNING.
- **Inner def `_status_badge(job_id)`:** returns job status string (or UNKNOWN).
- **Inner def `_show_job_error(job_id)`:** when FAILED, shows stored traceback + last logs in an expander.
- **`main()`**
- **Technical:** Streamlit entrypoint: sets page config, initializes singletons, renders mode selector + date range + chart options + runtime config expander, optionally fetches USREC recession shading data, then calls `_render_query_and_submit` and `_render_results`.
- **Intuitive:** “Build the whole app page.”
- **Gotchas:** Validates `end_date >= start_date` and calls `st.stop()` on invalid input; depends on optional deps for some features (plotly, providers, models).

3.2 `scripts/e2e_smoke.py` (smoke test entrypoint)

- **`main()`**
- **Technical:** Creates a temporary working dir, initializes `SQLiteStorage` + `JobRunner`, submits provider fetch jobs (market + news), waits/polls for SUCCEEDED, asserts artifacts exist and contain expected columns, then submits recipe jobs (SMA on market, inflation_yoy + AR1 on FRED) and asserts outputs; finally shuts down the runner executor.
- **Intuitive:** A quick “does the whole pipeline work end-to-end with real network providers?” check.
- **Gotchas:** Requires network + optional deps (`.[market]`, `.[news]`, `.[macro]`, `.[econ]`); polling uses tight sleeps and a small job list limit; uses private attribute `runner._executor` for shutdown.
- **How to run:** `python scripts/e2e_smoke.py`

3.3 `src/finrec/config.py` (configuration)

- **`load_config()`**

- **Technical:** Calls `dotenv.load_dotenv(override=False)` then reads `FINREC_DB_PATH` and `FINREC_RESULTS_DIR` with defaults (`data/finrec.db`, `data/results`); creates parent directories; returns `AppConfig(db_path, results_dir)`.
- **Intuitive:** Central place to decide “where do we store the SQLite DB and output CSVs?”
- **Gotchas:** `override=False` means existing environment variables win over `.env`; directory creation happens at load time.

3.4 `src/finrec/jobs/runner.py` (job execution)

- `JobContext.log(level, message)`
- **Technical:** Delegates to `SQLiteStorage.append_log(job_id, level, message)`.
- **Intuitive:** A tiny logger that routes job logs into SQLite.
- **Gotchas:** Logging is synchronous (each call writes to SQLite).
- `JobRunner.__init__(storage, results_dir)`
- **Technical:** Stores `storage`, normalizes `results_dir` to a `Path`, and creates a `ThreadPoolExecutor(max_workers=2)` to run jobs concurrently.
- **Intuitive:** Sets up the “background worker” pool.
- **Gotchas:** Only 2 workers—fast enough for a toy app; slow providers/models can block throughput.
- `JobRunner.submit(kind, provider_kind, provider_id, request, fn) -> job_id`
- **Technical:** Creates a job row in SQLite (`status=QUEUED`), then submits an inner `_run()` to the thread pool:
- set status `RUNNING`
- call `df = fn(ctx)`
- if `df` is not `None`, write `results_dir/{job_id}.csv` and store `output_path`
- set status `SUCCEEDED`; on exception set `FAILED`, store traceback in `jobs.error`, and log `ERROR` lines.
- **Intuitive:** “Run this task later and keep track of its progress + output.”
- **Gotchas:** Artifact writing is “best effort”—if `df.to_csv` fails, the whole job fails; `request` is persisted as JSON for debugging/repro.
- **Inner def `_run()`:** the actual job lifecycle wrapper that provides standardized status/log/error handling.

3.5 `src/finrec/storage/sqlite.py` (job persistence)

- `_utc_now_iso()`
- **Technical:** Returns a UTC ISO timestamp to seconds (e.g., `2026-02-15T12:34:56+00:00`).
- **Intuitive:** Standard timestamp format for jobs/logs.
- **Gotchas:** Second-level precision (good enough for UI).
- `SQLiteStorage.__init__(db_path)`
- **Technical:** Stores the path to the SQLite database file.
- **Intuitive:** “Point storage at a DB file.”
- **Gotchas:** Doesn’t create schema; call `init_schema()` first.
- `SQLiteStorage._connect()`

- **Technical:** Opens a sqlite3 connection with `check_same_thread=False` and `row_factory=sqlite3.Row`.
- **Intuitive:** “Give me a connection that can be used from job threads.”
- **Gotchas:** SQLite has concurrency limits; this app mitigates by small `max_workers`.
- `SQLiteStorage.init_schema()`
- **Technical:** Creates `jobs` and `job_logs` tables if missing. If `jobs` exists but its column list doesn’t match the expected schema, it drops both tables and recreates them.
- **Intuitive:** “Make sure the database has the right tables to track jobs.”
- **Gotchas:** Schema mismatch triggers a destructive reset (drops existing job history/logs).
- `SQLiteStorage.create_job(job_id, kind, provider_kind, provider_id, request)`
- **Technical:** Inserts a new row into `jobs` with timestamps, `status='QUEUED'`, and `request_json=json.dumps(request_json, sort_keys=True)`.
- **Intuitive:** Creates a new “job record” before work starts.
- **Gotchas:** `request` must be JSON-serializable.
- `SQLiteStorage.set_status(job_id, status)`
- **Technical:** Updates `jobs.status` and `updated_at`.
- **Intuitive:** Moves jobs through QUEUED → RUNNING → SUCCEEDED/FAILED.
- **Gotchas:** Status is free-form text; conventions are enforced by `JobRunner`.
- `SQLiteStorage.set_output_path(job_id, output_path)`
- **Technical:** Stores the artifact path (string) in `jobs.output_path`.
- **Intuitive:** Links a job to its output CSV file.
- **Gotchas:** Path is stored as text; file might be moved/deleted later.
- `SQLiteStorage.set_error(job_id, error)`
- **Technical:** Stores an error string (often exception + traceback) in `jobs.error`.
- **Intuitive:** Makes failures explainable in the UI.
- **Gotchas:** Can be large; UI truncates when displayed.
- `SQLiteStorage.append_log(job_id, level, message)`
- **Technical:** Inserts one row into `job_logs` with timestamp, level, message.
- **Intuitive:** Keeps a timeline of what the job did.
- **Gotchas:** Frequent logging can be slow (SQLite writes); keep messages lightweight.
- `SQLiteStorage.list_jobs(limit=50) -> list[JobRow]`
- **Technical:** `SELECT * FROM jobs ORDER BY created_at DESC LIMIT ?` and maps rows to `JobRow` dataclass instances.
- **Intuitive:** “Show me recent jobs.”
- **Gotchas:** Limit can hide older job ids referenced elsewhere (if callers use too-small limits).
- `SQLiteStorage.list_logs(job_id, limit=200) -> list[LogRow]`
- **Technical:** Fetches job logs ordered by insertion id ascending.
- **Intuitive:** “Show me what happened during job X.”
- **Gotchas:** UI often shows only the tail; increase `limit` for deeper debugging.

3.6 src/finrec/providers/** (data providers)

3.6.1 providers/utils/optional.py (optional dependency gating)

- `optional_import(module_name)`

- **Technical:** `importlib.import_module`; returns module or `None`; catches all exceptions.
- **Intuitive:** “Try to import it, but don’t crash if it’s missing.”
- **Gotchas:** Catches *any* exception (not just `ImportError`), so real import-time bugs can be masked.
- **`require_optional(module_name, extra_hint)`**
- **Technical:** Calls `optional_import`; if missing, raises `ImportError` with an install hint like `pip install -e "[dev,market]"`.
- **Intuitive:** Turns “module not installed” into a friendly, actionable error.
- **Gotchas:** The extras list is heuristic: always includes `dev` plus the provided hint.

3.6.2 providers/registry.py (dynamic provider registration)

- **`ProviderRegistry._key(kind, provider_id)`**
- **Technical:** Builds stable dict key as `"{kind}:{provider_id}"`.
- **Intuitive:** Namespaces provider ids by type (market vs macro vs news).
- **Gotchas:** `provider_id` must be unique *within* a kind.
- **`ProviderRegistry.register(provider)`**
- **Technical:** Inserts provider into internal dict by key derived from `provider.meta`.
- **Intuitive:** “Add this provider implementation to the app.”
- **Gotchas:** Registration overwrites silently if keys collide.
- **`ProviderRegistry.list(kind)`**
- **Technical:** Filters registered providers by `meta.kind`.
- **Intuitive:** “What market providers are available right now?”
- **Gotchas:** Order is dict iteration order (insertion order).
- **`ProviderRegistry.get(kind, provider_id)`**
- **Technical:** Lookup by key; raises `KeyError` if missing.
- **Intuitive:** “Give me the specific provider implementation requested by UI.”
- **Gotchas:** The app relies on `get_registry()` having registered providers first.
- **`_try_register_real_providers(reg)`**
- **Technical:** Best-effort registration: checks optional imports (`yfinance/requests/pandas_datareader`) and conditionally registers `YFinanceProvider`, `FMPPProvider`, `FREDProvider`, `GDELTProvider`; wraps each block in `try/except` and never raises.
- **Intuitive:** Providers “appear” only if you installed the corresponding extras.
- **Gotchas:** Swallows exceptions, so a broken provider import can silently vanish from the UI.
- **`get_registry()`**
- **Technical:** Lazy-initializes a module-global registry singleton and calls `_try_register_real_providers`.
- **Intuitive:** “One provider registry for the whole app.”
- **Gotchas:** Registry is cached; changing installed deps mid-run won’t update the registry without restart.

3.6.3 providers/market/yfinance.py (Yahoo Finance via yfinance)

- **`_throttle_yfinance(min_interval_s=3.0)`**
- **Technical:** Global lock + last-request timestamp; sleeps to enforce minimum spacing between requests across threads.
- **Intuitive:** Reduces rate-limit errors by avoiding bursts.
- **Gotchas:** Process-wide throttle only (doesn’t coordinate across machines/processes).
- **`YFinanceProvider.fetch(request, ctx)`**

- **Technical:** Reads `symbol`, `interval`, and either a date range (`start_date/end_date`) or a fallback `n`; calls `Ticker(symbol).history(...)` with `retry/backoff` on `YFRateLimitError`; normalizes output into columns like `date`, `symbol`, `open/high/low/close/volume`.
- **Intuitive:** “Download price bars for a ticker and return a clean dataframe.”
- **Gotchas:** Yahoo throttling is common; intraday vs daily can change the timestamp column name; backoff can make jobs slow.

3.6.4 providers/market/fmp.py (Financial Modeling Prep, with fallback)

- `_is_fmp_premium_symbol_error(status_code, body)`
- **Technical:** Detects a common free-tier failure mode (HTTP 402 and message containing subscription/premium hints).
- **Intuitive:** Recognize “this ticker needs a paid FMP plan.”
- **Gotchas:** String-matching heuristic; could miss new message formats.
- `FMPProvider.fetch(request, ctx)`
- **Technical:** Requires `requests` and `FINREC_FMP_API_KEY`; pulls daily EOD history from the “stable” endpoint; supports date-range mode or `tail(n)` fallback; parses JSON list/dict shapes; normalizes to `date`, `symbol`, and numeric OHLCV; on premium-symbol error, falls back to `YFinanceProvider.fetch`.
- **Intuitive:** Prefer a paid API (more stable) but gracefully degrade to Yahoo when a ticker isn’t covered.
- **Gotchas:** Only supports daily bars (ignores non-1d intervals); missing API key is a hard error; fallback requires `yfinance` installed.

3.6.5 providers/macro/fred.py (FRED via pandas-datareader)

- `FREDProvider.fetch(request, ctx)`
- **Technical:** Requires `pandas_datareader.data`; reads `series_id` and either date range or fallback `n`; calls `DataReader(series_id, "fred", start, end)`; outputs long-form dataframe with `date` (string), `series_id`, `value`.
- **Intuitive:** “Get a macro series from FRED and return it in a consistent schema.”
- **Gotchas:** FRED has no future data (`end_date` is clamped to today); missing data raises; output is long-form (not wide).

3.6.6 providers/news/gdelt.py (GDELT doc API)

- `_gdelt_dt(d, is_end)`
- **Technical:** Formats a date into GDELT’s YYYYMMDDHHMMSS with start-of-day or end-of-day time.
- **Intuitive:** Converts “date range” into GDELT’s timestamp parameters.
- **Gotchas:** Day-level granularity only.
- `_normalize_gdelt_query(q)`
- **Technical:** Trims query; wraps top-level OR groups in parentheses if not already.
- **Intuitive:** Makes simple OR queries valid under GDELT syntax rules.
- **Gotchas:** Intentionally minimal to avoid breaking advanced user queries.
- `_append_sourcelang_filter(query, languages)`
- **Technical:** Appends `sourcelang:<lang>` filters; OR-groups multiple languages.
- **Intuitive:** “Restrict results to English (or a set of languages).”
- **Gotchas:** Language tokens must match GDELT’s expected names.

- `_throttle_gdelt(min_interval_s=5.0)`
- **Technical:** Global lock + last-request timestamp; enforces ≤ 1 request per 5 seconds across threads.
- **Intuitive:** Plays nicely with GDELT’s published rate limit.
- **Gotchas:** Still subject to 429s if other clients share your IP or limits change.
- `GDELTProvider.fetch(request, ctx)`
- **Technical:** Requires `requests`; validates `query`, `start_date`, `end_date`; builds request for `mode=ArtList` with `maxrecords $\leq$ 250`; retries on 429 with waits; handles occasional “status=200 but non-JSON body” by retrying; outputs dataframe with article metadata (`ts`, `date`, `query`, `language`, `title`, `snippet`, `source`, `url`).
- **Intuitive:** “Search recent articles about your topic and return a table you can read/score.”
- **Gotchas:** `maxrecords` is capped; titles may be missing and get filtered out; API reliability varies.

3.7 src/finrec/ui/**, src/finrec/viz/**, src/finrec/datasets/**

3.7.1 ui/pipelines.py (turn providers/recipes into jobs)

- `submit_provider_fetch(runner, kind, provider_id, request) -> job_id`
- **Technical:** Looks up provider from `get_registry().get(kind, provider_id)`; defines `job_fn(ctx)` that logs start/end and calls `provider.fetch(request, ctx)`; submits through `runner.submit(kind='provider_fetch', ...)`.
- **Intuitive:** “Run a provider fetch in the background and store its CSV artifact.”
- **Gotchas:** Provider selection must exist in the registry; request schema is provider-specific.
- **Inner def `job_fn(ctx)`:** wraps the provider call with standardized logging and returns the dataframe artifact.
- `submit_recipe_run(runner, input_job_id, input_path, recipe_id, params) -> job_id`
- **Technical:** Fetches recipe from `get_recipe_registry().get(recipe_id)`; defines `job_fn(ctx)` that loads the input artifact CSV, calls `recipe.run(df_in, params, ctx)`, returns output df; submits through `runner.submit(kind='recipe_run', provider_kind='recipe', ...)`.
- **Intuitive:** “Take the output of one job and run a transformation/model on it.”
- **Gotchas:** Assumes `input_path` points to a readable CSV; recipe expects specific columns depending on type.
- **Inner def `job_fn(ctx)`:** enforces the ‘load input $\rightarrow$ run recipe $\rightarrow$ return df’ pattern.
- `get_jobs_by_id(storage, limit=1000) -> dict[job_id, JobRow]`
- **Technical:** Calls `storage.list_jobs(limit=limit)` and builds a dict keyed by `job_id`.
- **Intuitive:** Makes it easy to look up job rows repeatedly during rendering.
- **Gotchas:** Limit applies—older jobs may not be present.

3.7.2 ui/timeseries_catalog.py (curated option lists)

- `_build_options(catalog, by_id, category_priority, recent_ids, custom_ids) -> list[SeriesOption]`
- **Technical:** Produces an ordered list: (1) recent ids (preserving recency), (2) custom ids not in catalog, (3) remaining catalog sorted by (category priority, per-item priority, id).
- **Intuitive:** “Put what you used recently at the top, but still offer a curated list.”
- **Gotchas:** Unknown ids become `SeriesOption(id, id, 'Other', 99)`.
- `build_finance_options(recent_ids, custom_ids)`
- **Technical:** Calls `_build_options` with the finance catalog + priorities.

- **Intuitive:** Builds the ticker dropdown list.
- **Gotchas:** Assumes ids are ticker-like (uppercased earlier).
- `build_econ_options(recent_ids, custom_ids)`
- **Technical:** Calls `_build_options` with the econ catalog + priorities.
- **Intuitive:** Builds the FRED series-id dropdown list.
- **Gotchas:** Same “unknown ids become Other” behavior.

3.7.3 viz/plotly_timeseries.py (Plotly figure construction)

- `build_recession_intervals(usrec_df) -> list[(start,end)]`
- **Technical:** Accepts wide (date,USREC) or long (date,value) recession indicator; finds contiguous periods where indicator == 1; returns inclusive (start,end) timestamp tuples.
- **Intuitive:** Converts the “USREC = 1 means recession” series into shaded blocks for charts.
- **Gotchas:** Requires clean date parsing; value coercion treats only exact 1 as recession.
- `plot_timeseries(df, left_cols, right_cols=None, ..., recession_intervals=None, log_y_left=False, log_y_right=False) -> Figure`
- **Technical:** Validates inputs; coerces x to datetime when possible (keeps numeric x for IRFs); plots left series (and optional right series on secondary axis); applies unified hover, axis labels, optional log scaling; if `recession_intervals` and x is datetime, adds shaded vrects.
- **Intuitive:** A reusable “nice chart” builder for both time-series and numeric-horizon plots.
- **Gotchas:** Requires optional Plotly deps (`. [dev]`); date coercion can drop rows; `right_cols` are filtered to avoid duplicates.

3.7.4 datasets/merge.py (artifact merging)

- `build_merged_dataset(inputs, merge_how='outer'|'inner', ffill=False, log=None) -> pd.DataFrame`
- **Technical:** For each input spec (CSV path, date_col, value_col, alias): loads CSV, coerces date/value, keeps one row per date, merges sequentially on `date`; optionally forward-fills after merge.
- **Intuitive:** “Stitch many single-series artifacts into one analysis dataset.”
- **Gotchas:** Schema must match the spec; forward-fill changes interpretation; duplicate dates are dropped per input.
- **Inner def `_log(level,msg)`:** optional callback wrapper used to log progress to job logs.

3.8 src/finrec/recipes/** (transformations and models)

3.8.1 Common recipe contract (what every recipe does)

- **Input:** a dataframe loaded from a prior job’s artifact (usually a provider output).
- **Params:** a dict (stringly-typed in UI) controlling columns/windows/hyperparameters.
- **Output:** a dataframe that will be written as a new CSV artifact.
- **Side effects:** should only log via `ctx.log(...)`; no global state except where explicitly cached (FinBERT pipeline).

3.8.2 recipes/registry.py (recipe registration)

- `RecipeRegistry.register(recipe)`
- **Technical:** Stores recipe object in `_recipes` keyed by `recipe.meta.id`.
- **Intuitive:** “Make this recipe selectable/runnable.”

- **Gotchas:** Overwrites silently if ids collide.
- **RecipeRegistry.list()**
- **Technical:** Returns list of recipe instances.
- **Intuitive:** “Which recipes exist?”
- **Gotchas:** Ordering follows insertion order in `get_recipe_registry()`.
- **RecipeRegistry.get(recipe_id)**
- **Technical:** Dict lookup; raises `KeyError` if missing.
- **Intuitive:** “Give me the implementation for recipe X.”
- **Gotchas:** Recipe ids must match what UI submits.
- **get_recipe_registry()**
- **Technical:** Lazy singleton; instantiates and registers all built-in recipes (transforms, indicators, macro, econ, news, forecasting).
- **Intuitive:** Central “menu” of capabilities.
- **Gotchas:** Some recipes require optional deps at runtime; they are still registered, but will error with `require_optional(...)` if deps aren’t installed.

3.8.3 `recipes/forecast/_utils.py` (forecasting utilities and standardized schema)

- **_parse_dates(df, date_col)**
- **Technical:** Parses datetimes with UTC, errors->NaT; requires at least one valid datetime; returns timezone-naive UTC timestamps for arithmetic.
- **Intuitive:** “Get a clean timeline from the dataframe.”
- **Gotchas:** If all dates fail to parse, forecasting aborts early.
- **_infer_step(dates)**
- **Technical:** Uses median spacing to infer a future-step cadence: month (~30d), quarter (~90d), year (~365d), week (~7d), else rounded day count.
- **Intuitive:** “How far apart are observations—daily? monthly? quarterly?”
- **Gotchas:** Heuristic; irregular time series can infer weird steps.
- **_make_future_dates(last_date, horizon, step)**
- **Technical:** Adds `step` repeatedly to generate `horizon` timestamps after `last_date`.
- **Intuitive:** Builds the forecast x-axis.
- **Gotchas:** `DateOffset(months=1)` respects calendar months (not fixed days).
- **_coerce_numeric(series, name)**
- **Technical:** `pd.to_numeric(..., errors='coerce')`; errors if all NaN.
- **Intuitive:** Ensure the target is actually numeric.
- **Gotchas:** Mixed strings become NaN and may shrink usable sample.
- **build_target_series(df, date_col, y_col, transform='none'|'log_return') -> ForecastFrame**
- **Technical:** Sorts by date; builds numeric y; optionally transforms to log returns; returns a `ForecastFrame` containing standardized df with `date` and `y`, plus `date_index` and the `target` label.
- **Intuitive:** “Turn raw columns into the exact series we want to forecast.”
- **Gotchas:** Log returns require positive values; nonpositive values become NaN and can reduce sample size.
- **split_train_test(dates, y, test_size)**
- **Technical:** Supports `test_size` as fraction or int; enforces minimum train length; returns (`train_dates`, `train_y`, `test_dates`, `test_y`).
- **Intuitive:** Split history into “fit” vs “evaluate” segments.
- **Gotchas:** Small samples are rejected (`n<10`).

- `build_forecast_output(...)` -> `pd.DataFrame`
- **Technical:** Builds a standardized long-form output with columns: `date`, `y`, `yhat`, `segment` in `{train,test,forecast}`, plus `method` and `target`; optionally adds `yhat_lower/yhat_upper` bands.
- **Intuitive:** Makes every forecast recipe produce the same shape so the UI can render them uniformly.
- **Gotchas:** Strict length checks to prevent silent misalignment; sorts by date and segment order.

3.8.4 Forecast recipes (all output the `build_forecast_output` schema)

- `NaiveForecastRecipe.run(df, params, ctx)** (id='forecast_naive')`
- **Technical:** Uses last non-NaN training value as constant prediction for both test and future horizon.
- **Intuitive:** “Tomorrow equals today” baseline.
- **Gotchas:** If training series becomes empty after preprocessing, it errors.
- `DriftForecastRecipe.run(df, params, ctx)** (id='forecast_drift')`
- **Technical:** Computes linear drift from first→last training value; forecasts by extending that line.
- **Intuitive:** Baseline with a trend.
- **Gotchas:** Needs ≥ 2 non-NaN training points.
- **Inner def `_path(steps)`:** generates the step-ahead drift trajectory from the last training value.
- `ETSForecastRecipe.run(df, params, ctx)** (id='forecast_ets')`
- **Technical:** Requires `statsmodels.tsa.holtwinters`; fits Holt-Winters exponential smoothing on training; forecasts `len(test)+horizon`.
- **Intuitive:** Smooths level/trend/seasonality and extrapolates.
- **Gotchas:** Needs ≥ 10 training obs; seasonality params must match cadence (e.g., 12 for monthly).
- `ARIMAForecastRecipe.run(df, params, ctx)** (id='forecast_arima')`
- **Technical:** Requires `statsmodels` SARIMAX; grid-searches small ARIMA(p,d,q) over AIC; forecasts `len(test)+horizon`; attempts confidence intervals via `conf_int(alpha=...)`.
- **Intuitive:** “Let the data pick a simple ARIMA structure, then project forward.”
- **Gotchas:** Needs ≥ 20 training obs; many orders can fail—logs warnings; CI may be unavailable.
- `_make_features(window, include_stats)`
- **Technical:** Converts a lag window into feature vector (raw lags + optional mean/std).
- **Intuitive:** “Turn last N values into model inputs.”
- **Gotchas:** Stats features can help linear models but may hurt others.
- `_build_supervised(y, lookback, include_stats) -> (X,y)`
- **Technical:** Drops NaNs, builds row `\{(t)\}` from lags `\{([t-lookback, ..., t-1]\}\}` predicting `\{(y_t)\}`.
- **Intuitive:** Creates a regression dataset from a single time series.
- **Gotchas:** Needs enough history: `n > lookback + 5`.
- `_recursive_predict(model, init_window, steps, include_stats)`
- **Technical:** Predicts one step ahead repeatedly, appending predictions into the window to forecast multiple steps.
- **Intuitive:** “Use the model’s own output as the next input” multi-step forecasting.
- **Gotchas:** Error accumulation is common in recursive forecasts.
- `RidgeLagForecastRecipe.run(df, params, ctx)** (id='forecast_ridge_lags')`
- **Technical:** Requires `sklearn`; fits `StandardScaler + Ridge(alpha)` on lag features; uses recursive prediction for `len(test)+horizon`.

- **Intuitive:** A lightweight ML forecaster using recent history as features.
- **Gotchas:** Feature/target build drops NaNs; recursive strategy can drift.
- `RandomForestLagForecastRecipe.run(df, params, ctx)** (id='forecast_rf_lags')`
- **Technical:** Requires sklearn; fits `RandomForestRegressor`; forecasts via the same recursive routine.
- **Intuitive:** Nonlinear ML baseline for lag-based forecasting.
- **Gotchas:** Can be slower; still recursive multi-step.

3.8.5 Time-series transforms

- `LogReturnsRecipe.run(df, params, ctx)** (id='log_returns')`
- **Technical:** Computes $\log(p_t) - \log(p_{t-1})$ from `price_col`; warns if nonpositive values; writes to `out_col`.
- **Intuitive:** Converts prices into (approx) continuously-compounded returns.
- **Gotchas:** Nonpositive prices become NaN; diff shifts by one.
- `RollingZScoreRecipe.run(df, params, ctx)** (id='rolling_zscore')`
- **Technical:** Rolling mean/std (`ddof=0`) and z-score $(x - \mu) / \text{sd}$ with `min_periods` `window/3`.
- **Intuitive:** Measures “how unusual is the current value vs recent history?”
- **Gotchas:** When $\text{sd} \approx 0$, z can explode to inf/NaN .

3.8.6 Technical indicators

- `SimpleMovingAverageRecipe.run(df, params, ctx)** (id='sma')`
- **Technical:** Rolling mean over window of `value_col`; writes to `out_col`.
- **Intuitive:** Smooth trend line.
- **Gotchas:** Early rows have NaNs due to windowing.
- `ExponentialMovingAverageRecipe.run(df, params, ctx)** (id='ema')`
- **Technical:** Exponentially-weighted moving average with `span`; writes to `out_col`.
- **Intuitive:** Smoother that reacts faster to recent moves than SMA.
- **Gotchas:** Choice of `span` changes lag/responsiveness.
- `RSIRecipe.run(df, params, ctx)** (id='rsi')`
- **Technical:** Uses price differences; computes gains/losses; applies Wilder-style smoothing via $\text{EMA}(\alpha=1/\text{window})$; outputs 0–100 RSI.
- **Intuitive:** Momentum oscillator—how strong were recent up moves vs down moves?
- **Gotchas:** If $\text{avg_loss} \approx 0$, RSI can saturate near 100.
- `MACDRecipe.run(df, params, ctx)** (id='macd')`
- **Technical:** Computes $\text{EMA}(\text{fast}) - \text{EMA}(\text{slow})$ as MACD line; $\text{EMA}(\text{signal})$ of that as signal; histogram is difference; writes `{prefix}_line/_signal/_hist`.
- **Intuitive:** Trend/momentum indicator based on two moving averages.
- **Gotchas:** Defaults (12,26,9) assume daily-ish cadence; different frequencies may need retuning.
- `RollingVolatilityRecipe.run(df, params, ctx)** (id='rolling_vol')`
- **Technical:** Computes simple returns `pct_change()`, rolling std, optionally annualizes by $\sqrt{\text{periods_per_year}}$; writes to `out_col`.
- **Intuitive:** A rolling measure of “how bumpy is the series?”
- **Gotchas:** Annualization assumes correct `periods_per_year` for your frequency.

3.8.7 Macro transforms

- `InflationYoYRecipe.run(df, params, ctx)** (id='inflation_yoy')`

- **Technical:** `100*(value/value.shift(12)-1)`; replaces $\pm\text{inf}$ with NA; adds `out_col`.
- **Intuitive:** Year-over-year inflation rate from an index level.
- **Gotchas:** Assumes monthly cadence (12 lags); wrong frequency breaks interpretation.
- **InflationMoMAnnRecipe.run(df, params, ctx)** (id='inflation_mom_ann')**
- **Technical:** `1200*(log(value)-log(value.shift(1)))` for positive value; adds `out_col`.
- **Intuitive:** Annualized month-over-month inflation using log differences.
- **Gotchas:** Requires strictly positive series; assumes adjacent rows are months.
- **GrowthQoQAnnRecipe.run(df, params, ctx)** (id='growth_qoq_ann')**
- **Technical:** `400*(log(value)-log(value.shift(1)))` for positive value; adds `out_col`.
- **Intuitive:** Annualized quarter-over-quarter growth.
- **Gotchas:** Assumes adjacent rows are quarters.
- **SpreadRecipe.run(df, params, ctx)** (id='spread')**
- **Technical:** Computes `series_a - series_b` from two input columns; default output column `{a}_minus_{b}`.
- **Intuitive:** “Yield spread” / “difference between two rates/series.”
- **Gotchas:** Requires both columns already present (typically from a merged dataset).
- **RealRateRecipe.run(df, params, ctx)** (id='real_rate')**
- **Technical:** Computes `nominal_col - inflation_col`; writes to `out_col`.
- **Intuitive:** Ex-post real rate approximation.
- **Gotchas:** Units must match (both in percentage points, not decimals).

3.8.8 Econometrics

- **OLSRecipe.run(df, params, ctx)** (id='ols')**
- **Technical:** Requires `statsmodels`; parses `x_cols` comma list; coerces to numeric; drops NaNs; fits `OLS(y, add_constant(X))`; returns a tidy coefficient table with `coef`, `std_err`, `t`, `p_value`, and model stats (`nobs`, `r2`, `adj_r2`).
- **Intuitive:** Classic regression “what explains y?” with readable output.
- **Gotchas:** Needs enough complete rows after dropping NaNs; `x_cols` must exist as columns (usually from merged dataset).
- **AR1Recipe.run(df, params, ctx)** (id='ar1')**
- **Technical:** Requires `statsmodels`; builds lagged regression $y_t = c + \phi y_{t-1} + e_t$ via OLS; returns coefficient table like OLS.
- **Intuitive:** “How persistent is this series?”
- **Gotchas:** Needs ≥ 10 obs; results can be noisy for short windows.
- **LocalProjectionIRFRecipe.run(df, params, ctx)** (id='lp_irf')**
- **Technical:** Requires `statsmodels`; for horizons $h=0..H$: regress y_{t+h} on `shock_t` (+ optional controls), store shock coefficient and CI bounds using a normal critical value; returns rows with `horizon`, `irf`, `std_err`, `ci_low`, `ci_high`, `nobs`.
- **Intuitive:** Measures how y responds over time after a shock today, without specifying a full VAR.
- **Gotchas:** Needs enough data after shifting/dropna; CI uses normal approx (no `scipy`).
- **Inner def _zcrit(level):** maps common CI levels to z-scores (0.90/0.95/0.99) with 0.95 fallback.

3.8.9 News sentiment (FinBERT)

- `_get_pipeline()`

- **Technical:** Lazily caches a HuggingFace `transformers.pipeline("sentiment-analysis", model="ProsusAI/finbert")` in a module-global `_PIPELINE`.
- **Intuitive:** Load the model once; reuse it across jobs.
- **Gotchas:** Requires `transformers` + `torch`; model download can be slow on first run.
- `_label_to_signed_score(label, score)`
- **Technical:** Maps POS→+score, NEG→-score, else 0.
- **Intuitive:** Turn classification outputs into a single signed sentiment number.
- **Gotchas:** Neutral labels collapse to 0 regardless of confidence.
- `FinBERTDailySentimentRecipe.run(df, params, ctx)** (id='news_sentiment')`
- **Technical:** Builds text from title/snippet; scores in batches; assigns per-article signed score; aggregates by date to daily mean/sum + article count; computes positive/negative label shares.
- **Intuitive:** Convert a messy article list into a clean daily sentiment time series.
- **Gotchas:** Input must have `title` and a parseable timestamp/date; grouping by date drops intraday detail.
- **Inner def** `_share(df_g, needle)`: helper to compute label share within each day's group.